Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Работа со стеком

Вариант № 9

Студент: **Дудырев Д.С.**

Группа: **ИУ7-32Б**

Преподаватель: **Барышникова М.Ю.**

2025 г.

Отчет лаб. работа №4

Описание условий задачи

Реализовать операции работы со стеком, который
представлен в виде статического массива и в
виде односвязного списка, оценить преимущества и недостатки каждой
реализации, получить представление о механизмах выделения и освобождения
памяти при работе с динамическими структурами данных.
Задание по варианту: вывести развернутые строки в обратном порядке

Техническое задание

Исходные данные

Пользовательский ввод:

1. Режим работы – эмуляция стека или выполнение сравнения
2. Тип данных на которых работает стек – статический массив или список
3. Команды для выполнения – добавить элемент, удалить элемент, вывести развернутые строки в обратном порядке, посмотреть содержимое стека, посмотреть содержимое стека с адресами (только для списка), посмотреть истрию удалений (только для списка)

Выходные данные

- Вывод содержимого стека от первого добавленного до последнего, для стека на списке можно выводить адреса каждого элемента.
- Вывод развернутых строк в обратном порядке
- Вывод адресов из которых удалялись данные (только для стека на списке)
- Таблица результатов сравнения стека на статическом массиве и списке

Описание реализуемой задачи

Реализация добавления и удаления из стека элементов. Просмотр содержимого стека. Просмотр содержимого стека с адресами элементов, для стека на списке. Просмотр истории адресов используемых для удаленных элементов. Выполнение сравнения необходимого времени и памяти для работы стека на статическом массиве и списке.

Способы обращения к программе

Программа запускается из командной строки при помощи команды `app.exe`.

Возможные аварийные ситуации и ошибки пользователя

Аварийная ситуация	Код ошибки
Переполнение стека	10
Попытка достать элемент из пустого стека	11
Ошибка ввода	12
Пустая строка	13

Аварийная ситуация	Код ошибки
Строка длинее 63 символов	14
Номер команды вне доступного диапазона	15
Ошибка выделения памяти	17

Описание структуры данных

Хранение структур в программе

- Для статического массива используется массив определенный в функции `main`
- Для списка при добавлении или удалении происходит выделение или освобождение памяти.

Структуры

Стек основанный на статическом массиве

```
#define STACK_MAX_LEN 64

#define STR_MAX_LEN 64

#define str_t char*

typedef struct
{
    char (*down_p)[STR_MAX_LEN];  // Нижняя граница стека
    char (*sp)[STR_MAX_LEN];      // Текущий элемент
} my_stack_t;
```

- `down_p` – указатель на нижнюю границу массива стека
- `sp` – указатель на текущий элемент

Стек основанный на списке

```
typedef struct stack_list_t stack_list_t;
struct stack_list_t
{
    stack_list_t *prev;    // Предыдущий элемент стека
    char sp[STR_MAX_LEN];  // Текущий элемент
};
```

- `prev` – указатель на предыдущий элемент
- `sp` – значение текущего элемента

Описание алгоритма

Статический массив

Добавление элемента

```
stack.next — следующий элемент
stack.sp — указатель на текущий элемент
```

```
stack.up_p — указатель на конец массива
value — значение из последней ячейки

stack.sp = stack.next
Если stack.sp >= stack.up_p
    вернуть STACK_OVERFLOW
Конец Если

скопировать value в stack.sp
```

Удаление элемента

```
stack.prev — предыдущий элемент
stack.sp — указатель на текущий элемент
stack.down_p — указатель на начало массива
value — значение из последней ячейки

Если stack.sp < stack.down_p
    вернуть STACK_IS_EMPTY
Конец Если

скопировать stack.sp в value
stack.sp = stack.prev
```

Список

Добавление элемента

```
stack — указатель на текущий элемент
value — получаемо значение
new_node.value — значение нового элемента
new_node.prev — указатель на пред. элемент

скопировать value в new_node.value
new_node.prev = stack

stack = new_node
```

Удаление элемента

```
stack — указатель на текущий элемент
stack.prev — указатель на пред. элемент
value — получаемое значение

Если !stack
    вернуть STECK_IS_EMPTY

stack = stack.prev
```

Необходимые функции

Статический массив

`error pop(my_stack_t *stack, str_t value);` – удаляет элемент из стека на статическом массиве

- `stack` – указатель на стек
- `value` – значение, которое было удалено
  `error push(my_stack_t *stack, str_t value);` – добавляет элемент в стек на статическом массиве
- `stack` – указатель на стек
- `value` – значение, которое будет добавлено
  `error print_all_stack(my_stack_t *stack);` – выводит содержимое стека на статическом массиве
  `stack` – указатель на стек
  `error print_rev_str(my_stack_t *stack);` – выводит развернутые строки из стека на массиве
  `stack` – указатель на стек

Список

`stack_list_t* create_stack_node(stack_list_t *stack, str_t str);` – инициализация элемента стека на списке

- `stack` – указатель на стек
- `str` – значение в элементе стека
  `error pop_l(stack_list_t **stack, str_t value);` – удаление элемента стека на списке
- `stack` – указатель на указатель на стек
- `value` – получаемое значение
  `error push_l(stack_list_t **stack, str_t value);` – добавление элемента стека на списке
- `stack` – указатель на указатель на стек
- `value` – значение для добавления
  `error print_all_stack_list(stack_list_t *stack);` – вывод содержимого стека на списке
- `stack` – указатель на стек
  `error print_all_stack_list_address(stack_list_t *stack);` – вывод содержимого стека на списке с адресами по которым располагаются элементы
- `stack` – указатель на стек
  `error print_rev_str_l(stack_list_t **stack);` – вывод строк стека на списке в развернутом виде
- `stack` – указатель на указатель на стек
  `void free_stack_list(stack_list_t *stack);` – освобождение памяти выделенной для стека на списке
- `stack` – указатель на стек

История удалений

`del_history_arr *create_del_history_arr(void);` – выделение памяти для первых элементов
`error add_elem_to_history_arr(del_history_arr *arr, stack_list_t *value);` – добавление нового элемента в массив

- `arr` – указатель на массив
- `value` – адрес элемента, который нужно добавить в массив
  `error print_history_arr(del_history_arr *arr);` – вывод содержимого
- `arr` – указатель на массив
  `void free_history_arr(del_history_arr *arr);` – освобождение памяти
- `arr` – указатель на массив

Обработка строк

`void reverse_str(char *str, char *str_orign)` – копирует развернутую строку `str_origin` в `str`

- `str` – развернутая строка
- `str_origin` – исходная строка

Тесты

Негативные

Ввод	Код ошибки	Комментарий
-1	15	Команда вне доступного диапазона
h	15	Команда не число
4	15	Команда вне доступного диапазона
1 5	15	Команда вне доступного диапазона

Позитивные

В таблице приведены тесты при выборе статического массива, но аналогичные входные данные используются для тестирования стека на списке

Ввод	Вывод	Комментарий
1 1 1 123 4 0	123	Добавление одного элемента
1 1 1 123 1 456 4 0	456 123	Добавление нескольких элементов
1 1 1 123 1 456 1 789 2 4 0	456 123	Добавление и удаление не всех элементов
1 1 4 0	Стек пустой	Пустой стек сразу после начала работы программы

Ввод	Вывод	Комментарий
1 1 1 123 1 456 2 2 4 0	Стек пустой	Добавление и удаление всех элементов и вывод пустого стека

Оценка эффективности

Замер времени включает в себя следующие действия:

- Добавление всех элементов в стек
- Разворот всех элемен

N	Статический массив		Список		Результаты	
	Время, сек	Память, байт	Время, сек	Память, байт	Время	Память
10	0.000001	6400016	0.000001	720	32.65%	99.99%
20	0.000001	6400016	0.000003	1440	48.69%	99.98%
50	0.000002	6400016	0.000007	3600	66.00%	99.94%
100	0.000007	6400016	0.000015	7200	55.92%	99.89%
500	0.000028	6400016	0.000055	36000	49.42%	99.44%
1000	0.000042	6400016	0.000088	72000	52.71%	98.88%
3000	0.000118	6400016	0.000268	216000	56.05%	96.63%
5000	0.000196	6400016	0.000449	360000	56.37%	94.38%
8000	0.000315	6400016	0.000723	576000	56.43%	91.00%
10000	0.000394	6400016	0.000907	720000	56.58%	88.75%
30000	0.001297	6400016	0.002925	2160000	55.66%	66.25%
50000	0.002194	6400016	0.004924	3600000	55.44%	43.75%
70000	0.003137	6400016	0.007053	5040000	55.52%	21.25%
80000	0.003584	6400016	0.007992	5760000	55.16%	10.00%
90000	0.003990	6400016	0.009086	6480000	56.09%	-1.25%
100000	0.004492	6400016	0.009889	7200000	54.57%	-12.50%

Количество элементов	Коментарий к полученным данным	
	Стек на статическом массиве работает быстрее на	Стек на списке требует меньше памяти на
10	42%	97-99%
20	48%	
100-1.000	54-56%	
3.000-8.000		90-98%
10.000		
30.000		66%
50.000		43%
70.000		21%
80.000		10%

90.000		-1%
100.000		-12%

Фрагментация памяти

В результате тестирования программы в большенстве случаем была замечена дефрагментация памяти

Выводы

Стек, основанный на статическом массиве, дает преимущество по скорости на любом количестве элементов от 10 до 100.000. При увеличении количества элементов относительная скорость стека на статическом массиве увеличивается. Для 10 элементов данный стек быстрее на 42%, для 20 элементов — на 48%, для 100 и более элементов прирост скорости находиться в диапазоне от 54% до 56%. При этом используемая память стеком на односвязном списке требует на 97-99% меньше памяти при 10 элементах, 90-98% при 3.000-10.000 элементах. Стек, основанный на односвязном списке, при количестве элементов 10-1.000 требует на меньше 97-99%, при 3.000-10.000 требует на 90-98%, начиная с 30.000 до 80.000 требует от 66% до 10% меньше памяти. При 90.000 элементов односвязный список требует меньше на 1.25% больше памяти, значит, при заполненности статического массива на 90% эффективность использования памяти статического массива и односвязного списка уравнивается. И при увеличении количества элементов до максимального допустимого для статического стека в 100.000 статический стек занимает на 12% меньше.
Значит, при заполненности стека на 90-100% наиболее эффективной реализацией является статический массив, так как данная реализация позволяет уменьшить время работы и не менее эффективно расходовать память.

Контрольные вопросы

1. Что такое стек?

Специальный тип данных в котором, доступ к данным осуществляется по типу LIFO (last in firts out)

2. Каким образом и сколько памяти выделяется под хранение стека при различной его реализации?

- При использовании статического массива память выделяется сразу на все элементы стека
- При использовании динамического массива память выделяется во время выполнения программы, при необходимости выделяется дополнительная память или особождается не используема
- При использовании односвязного списка память выделяет или освобождает при добавлении или удалении каждого элемента

3. Каким образом освобождается память при удалении элемента стека при различной реализации стека?

- При использовании статического массива память освобождает после завершения программы
- При использовании динамического массива для стека освобождается данный массив, по указателю на 1-й элемент
- При использовании односвязного списка происходит по-элементное освобождение элементов стека по у текущего есть данные о предыдущем элементе

4. Что происходит с элементами стека при его просмотре?

После просмотра элементов в стеке буду располагаться в обратном порядке

5. Каким образом эффективнее реализовывать стек? От чего это зависит?

- Статический массив для стека эффективнее, если важна скорость работы программы. Но при заполненности статического массива 90-100% эффективность по памяти не меньше, чем у односвязнного списка
- Односвязнный список для стека эффективнее, если важно потребление памяти.